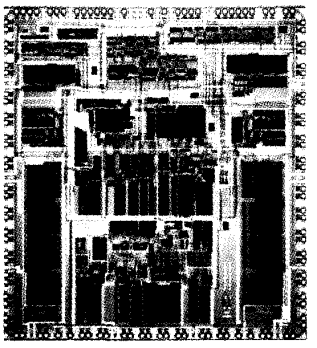
FEATURE



The 68040 Processor

Part 1, Design and Implementation

In the first of a two-part series, the design team explains its total approach and the workings of the integer and floating-point units.

Robin W. Edenfield
Michael G. Gallup
William B. Ledbetter, Jr.
Ralph C. McGarity
Eric E. Quintana
Russell A. Reininger

Motorola

The 68040 is a third-generation, full-32-bit microprocessor in the Motorola 68000 family. The 68040 integrates over 1.2 million transistors on one chip and can execute the complete 68020 microprocessor and 68882 floating-point coprocessor instruction sets. Its sustained performance level is four times that of the 68020 and 10 times that of the 68882 (all with the same clock frequency of 25 megahertz). Pipelined integer and floating-point execution units that operate concurrently with separate internal memory controllers and an autonomous bus controller contribute to this performance level. Physical caches of 4 Kbytes each for instruction and data reside on chip. Separate address-translation caches of 64 entries apiece operate in parallel with the instruction and data caches. This arrangement provides complete memory management in a virtual, demand-paged operating system. Here we present the design of this microprocessor and the trade-offs that we made in the process. (Figure 1 summarizes the processor.)

The major design goals in order of priority were

- compatibility,
- performance,
- schedule, and
- integration.

Because the 68040 joins a family of installed microprocessors, these fundamental goals primarily resulted from user input—and our perceived evolution of all microprocessors—over the term of the 68040 design.

We defined *compatibility* as the capability to execute all user-mode code without change. However, we realized that some changes to the supervisor code were inevitable due to the characteristics of the processor. Large physical cache, memory management unit (MMU) control, and chip integration all would require that the operating system "recognize" the difference between previous-generation processors and the 68040. This factor led us to allow deviations in the supervisor model that improved either performance or on-chip silicon requirements.

Market conditions required the processor to deliver four times the performance of the 68020 at the same clock speed, with working silicon implemented in 1989.

0272-1732/90/0200-0066$01.00 © 1990 IEEE

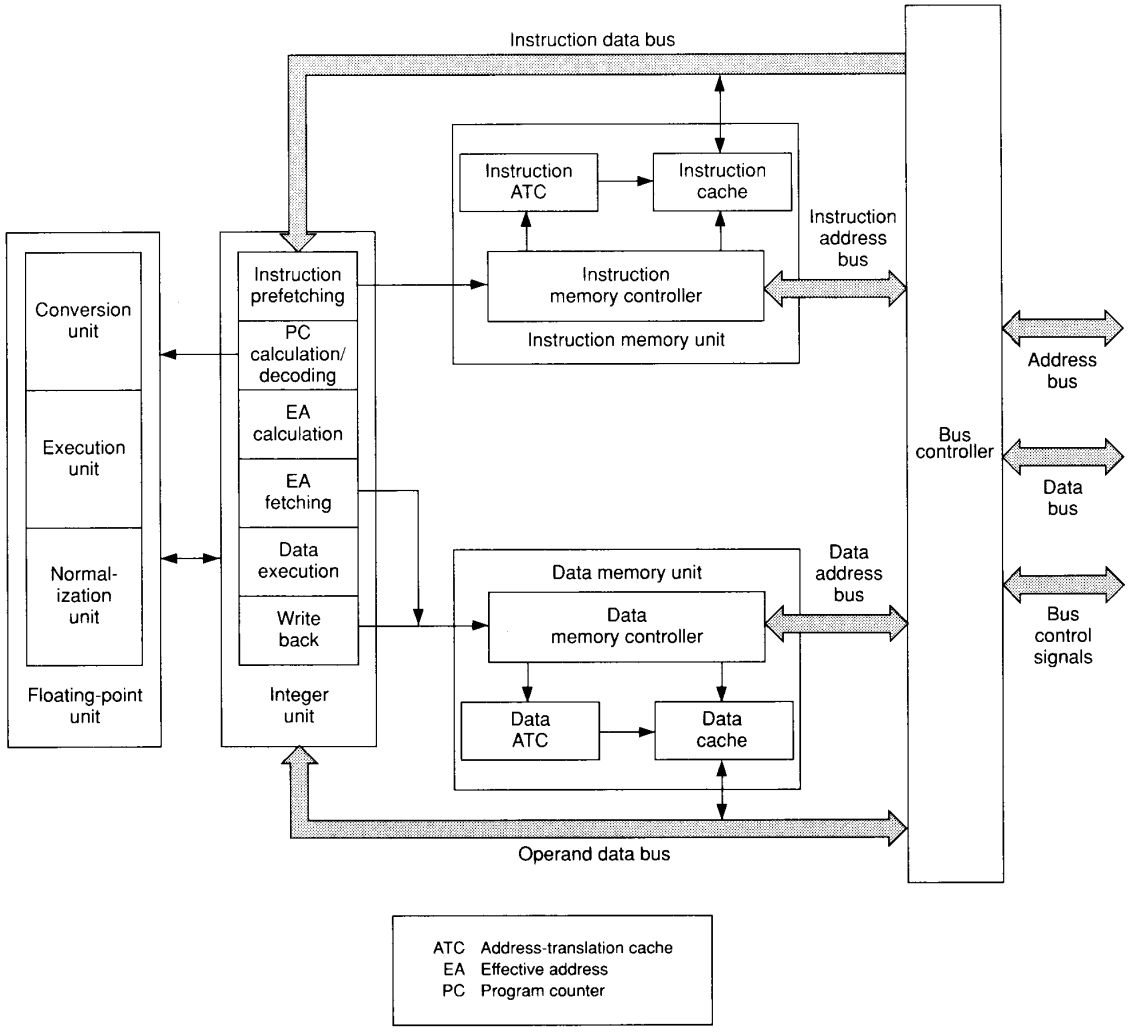


Figure 1. Block diagram of the 68040.

Our integration goal was to achieve the combined effect of separate 68000 family components: integer processing, on-chip floating point, and on-chip memory management capable of supporting a demand-paged operating system.

We followed a different process for this design from that traditionally performed at Motorola. The major changes were a heavy emphasis on early user input before and during the specification process and the use of functional simulation instead of breadboarding to verify the design. From the design goals, we presented an initial design specification to users. We worked with the users to refine the requirements and the architecture to produce the final specification.

Concurrently, other members of the design team studied the 68020 and 68882 instruction sets as used in system designs. Together we built a system to collect bus activity from a 68020 processor. With this system, we accumulated traces of over 60 million bus cycles of real application code in execution.

To aid in the analysis of the bus-cycle trace data, we created a high-level, statistical performance model. This model used parameters obtained from the traced bus cycles and disassembled instruction statistics. The model—along with a cache and an MMU simulator driven from the trace data—allowed us to quickly determine the effects of such design decisions as

Cache Modes

Data caches generally operate in one of two modes: write through or copy back.[1] On a cache hit, a write-though cache updates the data in the cache and writes the data to main memory via the external bus. The system keeps main memory up to date, or coherent, with the data in the cache. Because a write-through cache generally does not allocate on a write miss, the only way for new data to be placed in the cache is on a read miss.

For a write hit to a copy-back cache, the system updates the data in the cache, but does not update main memory. This situation creates "dirty" data in the cache. The only valid location for the data is in the cache since the memory version of the data is incorrect. Main memory is called incoherent when there is dirty data in the cache. Note that any read hit accesses the dirty data from the cache, not main memory, so that data integrity is preserved. The system only writes (pushes) a dirty cache entry to main memory when it needs to remove the entry to make room for another one. Copy-back caches generally allocate on a write miss.

- physical cache size and associativity,
- address-translation cache size and associativity,
- MMU table-search algorithm selection,
- branch methods and branch-target buffer inclusion,
- MMU page size,
- cache write methods, and
- bus clock speed.

Philosophy

The first major decisions we made were to optimize those instructions in the 68020 that executed most frequently and to allow other instructions to be implemented conveniently. We picked a reasonable subset of the 68020 instruction set based on trace data and user input. We examined those instructions and discovered that most of them would execute in one clock cycle if the IU were pipelined. (A clock cycle here refers to the input clock to the chip—which is also used by the system—running at 25 MHz.)

Along with this analysis, we decided that for performance reasons the overall cache architecture should continue to be based on the Harvard model (separate address and data spaces), as in the 68030. Further, we matched the integer-ALU cycle time to the cache-access time. This procedure allowed optimized instructions to access each cache once per clock cycle and fixed the peak instruction-execution rate to the ALU cycle rate. The entire internal and external memory design rested on these decisions.

External memory systems that feature either low cost or high performance were important to users. Consequently, the 68040 bus design maximized the performance available from low-cost memory systems (dynamic RAMs) while minimizing the degradation to high-performance designs (the fastest static RAMs). Therefore, we interfaced the bus to standard transistor-transistor logic (TTL) devices and simplified the protocol to minimize the latency from cache misses of required data.

Users predicted that most of the designs in the 68040-generation time frame would only use one processor for mainline computing. Consequently, we optimized both the internal cache operations and the external bus protocol for the uniprocessor case. We determined the data-cache write method based on principles of performance in uniprocessors versus requirements for shared memory access in multiprocessors. The uniprocessor case requires a copy-back write policy with read and write allocation for maximum performance and minimum bus utilization (see the accompanying box).

However, the multiprocessor case requires complicated bus and cache-state protocols with copy-back data. Also, several key users differed in the approach they wanted in a multiprocessor coherency scheme. These complications led us to add a write-through mode to the cache to specifically support multiprocessing. The write mode of the data cache—copy back or write through—can be set on a page basis by a field within the page descriptor used by the MMU. In write-through mode, the cache controller only allocates on a cache-read miss. Writes proceed directly to the bus and do not allocate a new entry in the cache. In copy-back mode, the cache allocates an entry in the cache on a miss and requests a line read on the bus, regardless of whether the integer unit requested a read or a write to memory.

Integer unit

The 68040 integer unit, or IU, executes the most common instructions in one clock cycle while maintaining user-code compatibility with the 68000 family. As discussed, only supervisor-mode instructions controlling the caches and MMUs differ from previous implementations.

To increase performance in the IU over previous-generation parts, the 68040 reduces both the number of ALU clock cycles per instruction and the ALU cycle time. For a 25-MHz 68020, the ALU cycles at 12.5 MHz, whereas the 25-MHz 68040 ALU cycles at 25 MHz.

Table 1.
A comparison of common effective-address instructions in the 68020 and 68040.

Instruction	Addressing mode	25-MHz clock cycles 68020	68040	Operation
Move	Rn,Rn	2	1	Register-to-register move
Move	<OEA>,Rn	6	1	Memory-to-register move
Move	Rn,<OEA>	6	1	Register-to-memory move
Move	<OEA>,<OEA>	8	2	Memory-to-memory move
Move	(An,Rn,d8),Rn	10	3	Memory-indexed source
Move	Rn,(An,Rn,d8)	6	3	Memory-indexed destination
Move Multiple	RL,<OEA>	$4 + 2n$	$2 + n$	Move multiple registers to memory
Move Multiple	<OEA>,RL	$8 + 4n$	$2 + n$	Move memory to multiple registers
Simple Arithmetic	Rn,Rn	2	1	Register-to-register arithmetic
Simple Arithmetic	Rn,<OEA>	6	1	Register-to-memory arithmetic
Simple Arithmetic	<OEA>,Rn	8	1	Memory-to-register arithmetic
Shifts	—	4	2	Shift 1 to 31 bits
Branch Taken	—	6	2	—
Branch Not Taken	—	4	3	—
Branch to Subroutine	—	6	2	—
Return from Subroutine	—	10	5	—

To reduce the number of clock cycles per instruction, we examined the previously mentioned trace data. From this data, we derived dynamic instruction frequency. Analysis of the instruction frequency led to the selection of a pipelined architecture to reduce load and store operations to one clock cycle. Pipelining allowed the 68040 to optimize the most common effective addressing modes:

- register direct, or Rn;
- address-register indirect, or (An);
- address-register indirect with postincrement, or (An)+;
- address-register indirect with predecrement, or −(An);
- address-register indirect with 16-bit offset, or (An,d16);
- absolute, or $Address; and
- immediate, or #Data.

These optimized effective-address (<OEA>) calculations and address fetches execute with no penalty for instructions such as logic, arithmetic, and data movement. Table 1 compares cycle times for the most common instructions with these effective addresses in the 68020 and 68040. (RL stands for multiple register list, and d8 refers to an 8-bit offset).

The more complex instructions that are not in the optimized set generally execute in less than half the number of 25-MHz clock cycles in a 68040 than they do in a 68020 or 68030.

Integer pipeline

The 68040 has a six-stage integer pipeline (Figure 1). Figure 2 on the next page details these stages:

- instruction-prefetching (IP),
- program-counter calculation and decoding (PCD),
- effective-address calculation (EAC),
- effective-address fetching (EAF) for operands,
- data-execution (DE), and
- write back (WB).

The implementation has three independent controllers: the PC controller for the IP and PCD stages; the effective-address (EA) controller for the EAC, EAF, and WB stages; and the DE controller for the DE stage. The PC controller is a finite state machine that relies on instruction-length and change-of-flow (branch, jump) decoding controls in the IP and PCD stages. The EA controller combines PLA decoding and microcode sequencing.

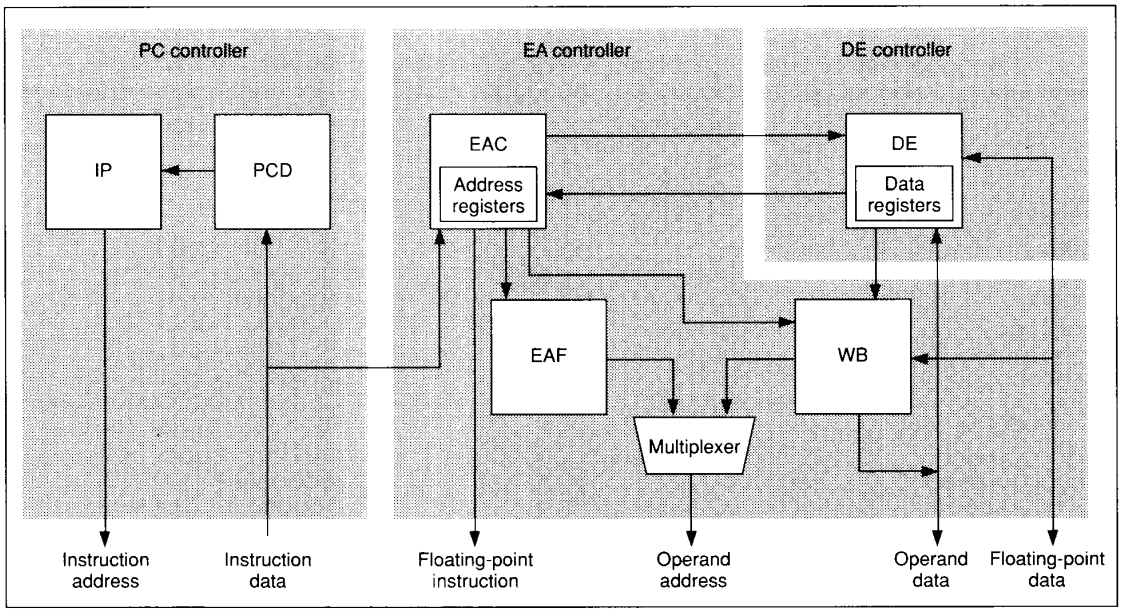


Figure 2. Integer pipeline of the 68040.

The first cycle of execution in the EAC stage receives control from a PLA decoder (not shown). A microcoded sequencer contained in the EAC stage subsequently controls this stage, and a second microcoded sequencer controls DE clock cycles. For most instructions, the EAC and DE stages work independently to allow maximum instruction-execution overlap. Extra cycles spent in each stage due to complex functions or cache misses can be fully overlapped by another stage. For complex instructions such as Move Multiple (MOVEM), the EAC and DE stages work together. The EAC stage generates a new load or store address each cycle, and the DE stage handles data movement to or from the register files.

The IP stage maintains a full 8-word (128-bit) instruction buffer to support execution of one-cycle, 3-word instructions indefinitely. As required, half of the instruction buffer is loaded with 64 bits from the instruction cache. The system can access the instruction cache every cycle to load the buffer.

The PCD stage selects 48 bits (3 sequential instruction words) from the buffer each cycle for instruction decoding and immediate data extraction. The 16-bit opcode and 6 bits from the first extension word provide for integer and floating-point instruction decoding. The 2 extension words provide for immediate and branch-displacement values. The PCD stage has separate instruction-length decoding to allow the PC to be incremented by 1, 2, or 3 words and to select a new 3-word instruction from the instruction buffer every clock cycle. The IU pipelines decoding for the EAC and DE stages and makes it available in the appropriate cycle.

The EAC stage calculates load-and-store effective addresses. It contains the address registers and controls them completely. This fact allows address-register loading and addition to complete early if the source operands are immediate values or other address registers. Early completion reduces stalls from address-register blockage for subsequent instructions that use the modified address register in an address calculation. The DE stage can update address registers if a source operand is either in memory (cache or main) or in a data register. Address-register updates by the DE stage result in only one clock cycle of address-register blockage if the next instruction uses the modified address register. Complex instruction addressing modes, such as memory indirection, require more than one clock cycle to complete. The EAC microcoded sequencer controls the second and succeeding cycles of these addressing modes.

The DE stage performs data-register operations and transfers data to the memory write-back buffer. The DE stage controls the data registers, condition codes, ALU, and barrel shifter (not shown). The DE also handles communication with other major functional units on the 68040 during exceptional conditions and memory-management table searches.

The EAF and WB stages provide for memory load-and-store accesses. These stages are controlled by combinational logic. When the EAC stage writes the

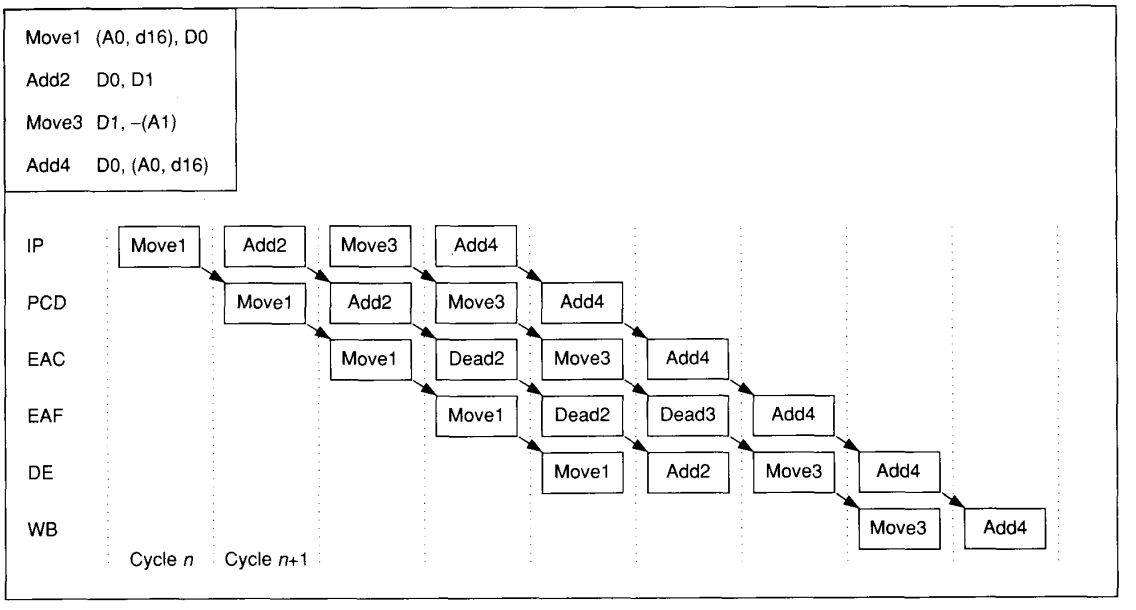


Figure 3. Integer instruction flow for typical optimized instructions in the 68040.

EAF address buffer, the EAF automatically initiates a load. A write-back address queue with an address register for each of the remaining stages of the pipeline facilitates store operations. For instructions that write to memory, the EAC loads the first register in this queue. When the DE stage completes execution of an instruction that requires a store operation, it loads the write-back data buffer. This action causes the WB stage to initiate a store to the cache and the queue to advance.

Since only one load or store operation can be initiated in each cycle, the EAF and WB stages must arbitrate for the data cache. Loads from the EAF stage normally have priority over stores from the WB stage except when the logical address for the load and store match. This match prevents a read-before-write hazard. The WB stage also gains priority if the DE requires a data load into the WB data buffer for a second store operation. A store can be buffered in the final WB stage indefinitely without impacting execution of subsequent instructions.

An underlying reason the 68040 architecture has dedicated EAC and EAF states is that 40 percent of all instructions have data-read accesses. With pipelining, most read accesses have no penalty. Dyadic instructions (Add, SUB, And, Or, EOR) that have a memory operand as either their source or destination execute in one clock cycle. Also, address register updates occurring in the EAC stage eliminate pipeline blockage for instructions with postincrements, predecrements, or immediate adds/loads to address registers.

Figure 3 demonstrates the pipeline operation by showing a typical sequence of instructions. Each column in the figure represents one 25-MHz clock cycle. The notation "dead" means that no useful activity occurs at that pipeline stage.

Branches

The 68040 improved branch performance relative to the 68020 and 68030 from six taken/four not-taken clock cycles to two taken/three not-taken cycles (ignoring cache-miss and cache-line-alignment effects). This performance occurs despite the deeper pipeline in the 68040. We optimized branches on the 68040 for the branch-taken case since trace data shows that 75 percent of all branch instructions (that is, Conditional Branch, Unconditional Branch, Branch to Subroutine, and Decrement and Branch) are taken. A dedicated branch adder in the PCD stage adds the branch displacement to the PC in every cycle. This addition is speculative as it occurs before the branch has been decoded. A dedicated branch-instruction decoder controls the processes of loading the result of the branch adder into the PC and starting a change-of-flow sequence in the PCD stage. If the instruction is not a branch, the IU ignores the result of the addition operation.

When a conditional change-of-flow sequence is started (a Conditional Branch or a Decrement Branch), the not-taken PC, the not-taken EAC decoded instruc-

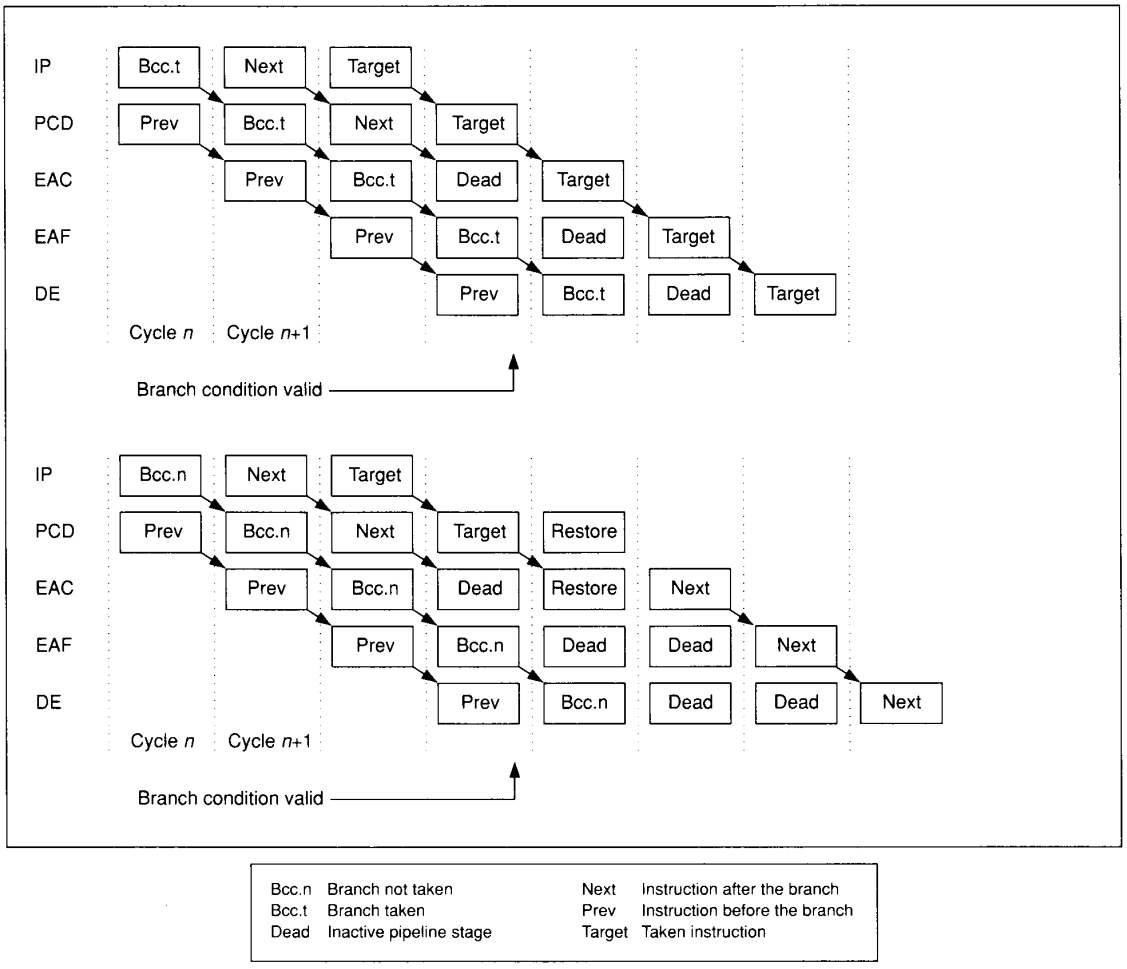


Figure 4. Integer instruction flow for taken (top) and not-taken (bottom) branch instructions.

tion, and the instruction buffer are saved in special shadow registers. As shown in Figure 4, the condition codes from the instruction preceding the branch are valid at the same time the first opcode stream has been decoded. This timing allows execution to start in the EAC stage for the target in the next clock cycle. If the condition is not taken, one cycle is required to recover the not-taken path from the shadow buffers.

We evaluated some design trade-offs for branch-instruction time minimization: 1) changing the architecture by adding delayed branches, 2) optimizing not-taken branches, and 3) adding a branch target buffer (BTB).[2] We rejected delayed branches because changes to the architecture would not help existing code. Optimizing not-taken branches created a problem of partially executed instructions in the EAC and EAF stages that had to be undone, which could result in clock-cycle penalties to the taken path. We rejected the inclusion of a 32-entry BTB based on issues of die size and circuit complexity. A one-cycle improvement in correctly predicted branches improved system performance by only 5.7 percent, which was not enough to justify the silicon-area investment. Also, system and circuit designs of the components to load, maintain, and clear the BTB were so complex they would have slowed the schedule.

Another interesting trade-off possibility existed. Should the PCD and EAC operations occur in one merged stage or in two separate ones? If the EAC and PCD states were united, fewer pipeline stages would separate them. This circumstance would allow an earlier resumption of prefetching for instructions that follow the instruction after the branch, lessening the chance of prefetching-induced stalls. However, merging the EAC and PCD stages would mean that far fewer

instructions would possess optimized (one-clock-cycle) effective-address calculations, since EAC-optimized decoding would only have part of one clock cycle to complete.

Since every prefetch operation obtains an entire half line of instructions, the IU prefetches enough of the instruction stream prior to the execution of the branch so that the extra stage imposes no penalty. We created a statistical model based on the average length of optimized instructions to determine whether the extra stage causes a stall. From it we found that the extra pipeline stage adds only 0.36 of a clock cycle to not-taken branches (making them 3.36 clock cycles). The increased number of optimized instructions provided by the extra pipeline stage more than compensates for this partial increase in clock cycles.

Integer exceptions

Exception processing in the 68040 differs from that of the 68020 and 68030 only in terms of memory-access faults. Traps, interrupts, and tracing are identical to the 68020 and 68030 programmer's model. For memory-access faults (bus-error and page), the existence of a restart exception model marks a change from the continuation exception model used in the 68010, 68020, and 68030. We selected the restart model because it needs fewer bytes to save exception information than a continuation model does (62 versus 250). A continuation model would have resulted in much slower exception stacking and return execution, as well as increased control and routing to access the state information on chip. However, the restart model requires the capability to "back out" of partially executed instructions.

When a memory-access error occurs (page fault or physical bus error), the 68040 stops execution of the current instruction and writes to the exception stack. This write contains the necessary state information for the operating system to correct the fault and return control to the current instruction stream. If an instruction has partially executed when the exception occurs, the state of the processor prior to starting the instruction is restored. The exception appears to the programmer's model to have occurred on an instruction boundary. The access-error stack frame contains

- the return instruction address,
- the faulting access address,
- the faulting access bus attributes,
- special case flags, and
- address and data for up to three pending memory writes.

The operating system completes pending writes.

The 68040 exception model has some special continuation cases. The Move Multiple instruction can corrupt address registers, data registers, and memory locations used in calculating the effective address. Restoring the state of the processor and memory prior to starting the instruction was not feasible, and simply restarting a MOVEM instruction could result in calculating a different effective address result. To solve this problem, an access error during a partially executed MOVEM stacks the calculated effective address and sets a flag on the stack frame. The Return from Exception instruction (RTE) reads the flag and the effective address on the stack frame. When the MOVEM instruction restarts, the IU skips the effective-address calculation and reads the effective address from the stack frame.

The other cases of continuation concern memory-access errors that are handled before instruction-generated exceptions. The special cases are trace exceptions, unimplemented floating-point exceptions, and floating-point postexceptions. When the RTE instruction executes after an access error, these continuation flags are tested. If they are set, the corresponding exception is immediately taken. The access error-stack frame contains the PC for the instruction causing the trace. It also contains the effective address needed by the exception handler for unimplemented floating-point exceptions and postexceptions.

Floating-point unit

The 68040 FPU is an on-chip functional unit that complements the IU. It is completely compatible with user object code in the 68881 and 68882 floating-point coprocessors and conforms to the IEEE 754 floating-point standard[3] via a software envelope. The FPU executes concurrently with the IU. The design goals for the FPU were compatibility and optimized double-precision performance.

The programmer's model for the 68040 FPU is identical to the one for the 68882. It consists of the following floating-point registers:

- eight 80-bit data,
- one 32-bit control,
- one 32-bit status, and
- one 32-bit instruction address.

The FPU supports a number of data formats:

- byte integer, (8 bits),
- word integer (16 bits),
- long-word integer (32 bits),
- single-precision binary real (32 bits),
- double-precision binary real (64 bits),
- extended-precision binary real (80 bits), and
- packed binary coded decimal (a software envelope handles this format).

The 68040 FPU implements a fundamental subset of

Table 2.
The 68040 FPU subset of 68882 instructions.

Mnemonic	Instruction
FADD	Add
FCMP	Compare
FDIV	Divide
FMUL	Multiply
FSUB	Subtract
FABS	Absolute Value
FSQRT	Square Root
FNEG	Negate
FMOVE	Move
FTST	Test
FBcc	Branch on Condition
FDBcc	Decrement and Branch on Condition
FScc	Set on Condition
FTRAPcc	Trap on Condition
FMOVEM	Move Multiple Registers
FSAVE	Save Context
FRESTORE	Restore Context

the 68882 instructions in hardware (see Table 2). All other 68882 instructions are emulated in software.

To change rounding modes or rounding precisions, 68882 software in the past had to read the control register, modify the appropriate bits, and place this value into the control register. Users and our own compiler designers requested that rounding precision control be implemented without the performance penalty of manipulating a control register. To this end, we added new variants to the basic floating-point instructions. These variants execute in the same way as the original instructions, except that the rounding precision specified by the control register is ignored and either forced to round to single- or double-precision, depending on the variant. For example, FADD creates a Floating-Point Add instruction, FSADD forces the result to be rounded to single precision, and FDADD forces the result to be rounded to double precision.

The 68040 does not impose an execution-time penalty for any rounding precisions or rounding modes. All calculations are always carried out to internal extended precision, with the result rounded to the destination precision.

Consistent with the user object-code compatibility goal, the FPU is truly separate from the IU. The exception model of user signal handlers already written for the 68882 coprocessor assumes this separation since the 68882 is separate from the CPU.

The FPU acts as a slave to the IU. The IU passes data to the FPU and queries for results. The FPU does not interrupt or make requests of the IU. This procedure exactly mimics the manner in which a 68882 communicates with its parent CPU. However, the 68040 FPU has the advantage of being on the same chip as its master. We used this advantage to let the FPU monitor the early stages of the IU instruction pipeline. By the time the IU is ready to pass or collect data, the FPU has predecoded the request and is ready to accept or send the data. Thus, we have minimized stalls in the IU caused by the FPU. The FPU accepts data and releases the IU even though the FPU may not be ready to start the accepted instruction (for example, register dependencies).

Floating-point pipeline

The FPU consists of a three-stage pipeline: the floating-point conversion unit (FCU), execution unit (FEU), and normalization unit (FNU). (See Figure 5.) The FCU is the only stage that communicates with the IU. It retrieves the operands, tags them (as normalized, denormalized, zero, infinity, or not-a-number), and converts them to extended precision. The FCU also performs an initial add or subtract operation involving the exponents to prepare for a possible alignment shift in the FEU. The FEU contains a mantissa 67-bit arithmetic unit (AU), a 67-bit barrel shifter, and a 64-bit × 8-bit multiplier array. The FEU performs the required calculations to produce an unrounded intermediate result. In fact, the FEU performs all of the fundamental floating-point algorithms. The FNU normalizes and rounds the result to the desired precision and also checks for exceptional conditions. The FNU then writes the result into a floating-point data register in the register file over a dedicated write-back bus (not shown).

With a 32-bit data port to the IU, transferring double-precision operands between the IU and FPU requires two clock cycles, while extended-precision transfers require three. The first stage of the FPU requires one additional cycle to detect IEEE 754 default cases (such as a floating-point multiplication of zero and infinity). For example, a dyadic instruction that has one operand in a floating-point register and the other operand from memory in double-precision format would occupy the first stage (the FCU) of the FPU for three clock cycles. However, once the double-precision data is received by the FCU, it immediately passes to the FEU so that the first clock cycle of execution in the FEU overlaps the third cycle of execution in the FCU. Although this overlap does not affect the fully pipelined execution time, it reduces the pipeline latency of an instruction by one clock cycle.

Fully pipelined Floating-Point Add and Subtract

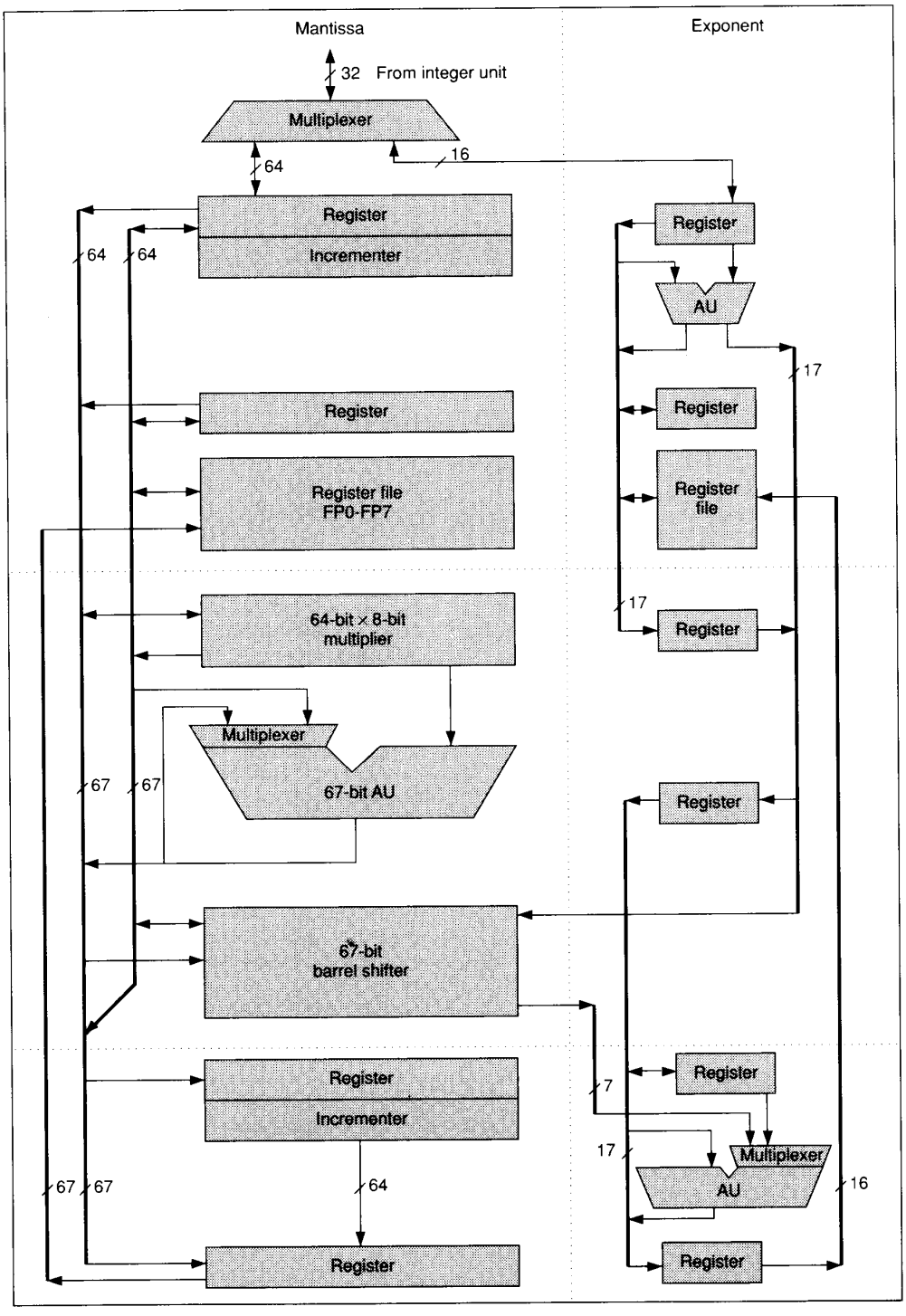


Figure 5. Floating-point unit of the 68040.

Table 3.
Floating-point instruction execution for the 68040 in clock cycles.

Instruction mnemonic	Source	Destination	FPU detail FCU	FEU	FNU	Fully pipelined, 25 MHz 68040	68882
FMOVE	Register	Register	2	0	0	2	21
FMOVE.D	Memory	Register	3	0	0	3	40
FMOVE.D	Register	Memory	3	0	0	3	44
FADD	Register	Register	2 (3)	3	2 (3)	3	21
FSUB	Register	Register	2 (3)	3	2 (3)	3	21
FMUL	Register	Register	2 (3)	5	2 (3)	5	76
FDIV	Register	Register	2 (3)	38	2 (3)	38	108
FSQRT	Register	Register	2 (3)	103	2 (3)	103	110
FADD.D	Memory	Register	2 (3)	3	2 (3)	3	75
FSUB.D	Memory	Register	2 (3)	3	2 (3)	3	75
FMUL.D	Memory	Register	2 (3)	5	2 (3)	5	95
FDIV.D	Memory	Register	2 (3)	38	2 (3)	38	127
FSQRT.D	Memory	Register	2 (3)	103	2 (3)	103	129

instructions execute in three clock cycles with double-precision operands. The FEU executes these instructions using the 67-bit barrel shifter to align the operands, the 67-bit AU to add the operands, and the same 67-bit barrel shifter for postnormalization—all within three clock cycles. This performance equates to over 8 million floating-point operations per second on double-precision operands (if one assumes hits in the data cache at 25 MHz).

A 64-bit × 8-bit hardware multiplier array in the FEU handles Floating-Point Multiply instructions. Performing a 64-bit × 64-bit, extended-precision, mantissa multiply operation requires eight passes through the array. During these passes, the outputs of the array are pipelined to the inputs (with the appropriate shift). We simply clock the array eight times to produce the result sums and carries. Since the multiplier array can be clocked twice per primary clock cycle, it takes four cycles to produce sums and carries. One more cycle is required to add these sums and carries to produce the final result. Therefore, fully pipelined FMUL instructions run five cycles each.

The Floating-Point Divide instruction uses a non-restoring shift-and-subtract algorithm to produce results. Two of the 66 quotient bits are produced every clock cycle (for a total of 33 clock cycles). Setup for the division loop requires two clock cycles. The remainder restore operation consumes up to three cycles. Thus, fully pipelined FDIV instructions run 38 cycles each.

Table 3 lists selected instruction timings for the FPU. The FCU passes data to the FEU as soon as it is available (two clock cycles for double precision). Similarly, results produced by the FNU are also available early (within two clock cycles). These overlaps help reduce instruction latency. Clock cycle times in parentheses are the total times a given stage is occupied by these operations. The fully pipelined instruction times assume optimized addressing modes and hits in the cache. Instruction extension .D indicates double precision.

Figure 6 shows a sample instruction sequence in the floating-point pipeline. Single-cycle integer instructions can execute in the empty slots of the IU without increasing the execution time. Floating-point results are available after the second cycle of execution in the FNU even though it is busy for three clock cycles.

Note that instructions not requiring the use of the mantissa adder, barrel shifter, or multiplier do not pass to the FEU. They execute completely in the FCU. Floating-Point Move, Absolute, and Negative instructions (FMOVE, FABS, and FNEG) execute entirely in the FCU. Also, the FCU alone handles several IEEE 754 default cases of other instructions (such as the floating-point add operation of zero plus zero). The consequence of this approach is that floating-point instructions can execute and complete in an out-of-order fashion for program sequences. This procedure is allowed as long as there are no register dependencies. The floating-point data registers are dually ported and fully scoreboarded to provide hardware protection for register dependencies during out-of-order instruction execution.

Exceptions

The programmer need not be concerned with the fact that floating-point instructions can complete nonse-

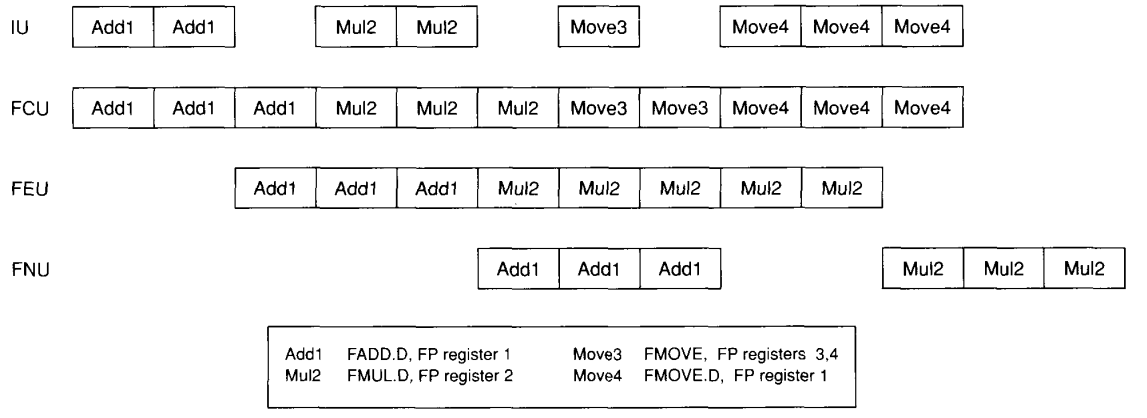


Figure 6. FPU instruction-flow diagram.

quentially. From the programmer's point of view, no reordering of instructions occurs. The system reports exceptions in the order that the programmer would expect, even when multiple exception-causing instructions execute out of order. This factor is important since user signal handlers exist for 68882 floating-point exceptions. (Floating-point exception handlers "jump" to user code so that the user can decide how to handle exceptions for a specific application.)

In-order exception reporting with out-of-order execution is possible because the floating-point exception model is a continuation model that contrasts with the restart model in the IU. Having two contrasting exception models on the same chip poses no additional programming constraints. The Save Context and Restore Context instructions behave in the same manner as in the 68882. The entire internal state of the FPU can be saved on the stack frame and restored with the FSAVE and FRESTORE instructions. FPU instructions that were saved on the stack frame simply continue where they left off once they are restored.

The 68040 FPU always minimizes the amount of state dumped onto the stack by the FSAVE instruction. The save operation of an idle FPU only stacks 1 long word. Consequently, during an FSAVE instruction, the FPU flattens its pipeline (if possible) to produce an idle state frame. This action typically results in faster context switches. If any instructions in the FPU pipeline generate an exception, the system stacks a longer "busy" state frame. However, the largest amount of state information that could be saved by the 68040 FPU is less than half of that stacked by the 68882. This reduced stack-frame size improves context-switching time.

Because it is separate from the IU, the FPU does not become directly involved in IU exception or interrupt processing. Only if the IU handler decides that the FPU is needed (such as for context switching) is an FSAVE or FRESTORE instruction necessary. The result is that the FPU can continue execution while the IU services an interrupt or integer exception.

Software envelope

All of the 68882 instructions that are not implemented on chip (like elementary functions) trap to the unimplemented instruction handler, where they are emulated in software. To ease the burden on the software emulator, the 68040 FPU creates an FSAVE state frame of only 11 long words. This state frame contains the operand (or both operands for dyadic instructions) already fetched and converted to extended precision. The state frame also contains tag information along with the instruction itself. If the emulator wants to report an exceptional condition, it sets the appropriate bits in the state frame and executes an FRESTORE instruction, which allows the 68040 hardware to handle the exception.

A software emulator of all unimplemented instructions is available from Motorola. The software emulation of elementary functions produces results that are more accurate than those of the 68882. The 68882 produces results accurate to 1.0 ulp (unit in the last place), double precision. On the other hand, 68040 software emulation results are accurate to 2.5 ulp, extended precision. The emulator's results are monotonic. Execution time of the software emulation for elementary functions including all trap overhead (running on a 25-MHz 68040) is 13 to 130 percent faster than the equivalent instructions on the 68882 running at 25 MHz. A user can further reduce execution time by recompiling with a compiler that replaces the unimplemented instructions with library calls.

In the first part of a two-part series, we have introduced the 68040, described design trade-offs, provided some insights into its implementation, and detailed the integer and floating-point units. In the June 1990 *IEEE Micro*, we will discuss the memory subsystem, the external bus, chip and board testing, and design-verification methods in the 68040.

Acknowledgments

We thank all of the people who have worked on the 68040 design and have helped to make its implementation possible.

References

1. A.J. Smith, "Cache Memories," *Computing Surveys*, Vol. 14, No. 3, Sept. 1982, pp. 473-530.
2. J.K.F. Lee and A.J. Smith, "Branch Prediction Strategies and Branch Target Buffer Design," *Computer*, Vol. 17, No. 1, Jan. 1984, pp. 6-22.
3. *ANSI/IEEE Standard 754-1985 for Binary Floating-Point Arithmetic*, IEEE Computer Society Press, Los Alamitos, Calif., 1985.


Edenfield


Gallup


Ledbetter


McGarity


Quintana


Reininger

Robin W. Edenfield is a principal staff engineer at Motorola and was primarily responsible for system design of the cache and memory management controllers for the 68040. He previously worked in the design, test, and production of microprocessor systems at several other companies. His current interests include microprocessor architecture and memory subsystems.

Edenfield received the BSEE degree from the Georgia Institute of Technology, where he was elected to Eta Kappa Nu.

Michael G. Gallup, with the microprocessor group at Motorola since 1981, is a circuit design engineer and a member of the technical staff for the high-end design group. His areas of interest include semiconductor devices, digital systems, and computer programming.

Gallup received the BSEE and MSEE degrees from the University of Nebraska at Lincoln. He is a member of the IEEE Computer and Circuits and Systems Societies.

William B. Ledbetter, Jr., is a principal staff engineer in high-end design at Motorola. He was responsible for the external bus specifications and protocol—as well as the system design of the bus controller—for the 68040. He also was a member of the architecture design team for internal memory. He previously designed graphics systems for Data General.

Ledbetter received the BSEE and MSEE degrees from Texas A&M University. He is a member of the IEEE Computer Society.

Ralph C. McGarity is a principal staff engineer and was the systems technical project leader for the 68040 project. He has spent seven years at Motorola designing 68000 family parts. His interests include microprocessor architecture design, implementation, and verification.

McGarity received the BSEE degree from Rice University and the MSEE degree from Carnegie Mellon University. He is a member of the IEEE, ACM SIGarch, Phi Beta Kappa, and Tau Beta Pi.

Eric E. Quintana has been with the microprocessor group at Motorola since 1984 as a staff engineer and is the architect for the floating-point section of the 68040. He previously worked as a design engineer on the 68020 microprocessor and later on the 68882 floating-point coprocessor.

Quintana received the BSEE degree from Texas A&M University.

Russell A. Reininger is a principal staff engineer for high-end microprocessors at Motorola. He was responsible for the system design of the integer unit in the 68040. Before joining Motorola, he worked for six years at Tracor Aerospace in Austin, Texas, where he designed radiation-hardened military computers.

Reininger received the BSEE degree from the University of Texas at Austin.

Readers can direct questions regarding this article to Ralph McGarity, Motorola Inc., M/S OE37, 6501 William Cannon Drive West, Austin, TX 78735-8598.